

CareBed 平台后端

CareBed 平台是面向医院共享陪护床业务的后端服务，基于 Spring Boot 3 与 Java 21 开发，覆盖从用户注册登录、设备运营、租借计费到报修、钱包、运营数据分析的完整能力。项目默认采用内存存储，便于原型演示与接口联调，后续可无缝迁移至持久化实现。

目录

1. [系统概览](#)
2. [架构与技术栈](#)
3. [环境准备](#)
4. [快速启动](#)
5. [配置说明](#)
6. [角色与权限矩阵](#)
7. [功能模块详解](#)
8. [核心业务流程](#)
9. [数据模型速览](#)
10. [API 参考](#)
11. [通知策略](#)
12. [测试与质量保障](#)
13. [运维与部署建议](#)
14. [常见问题](#)
15. [未来规划](#)

系统概览

- 支持陪护家属与运营管理员多角色登录与鉴权。
- 统一管理共享陪护床设备的注册、绑定、状态与故障。
- 覆盖租借、使用计费、归还、超时控制的全链路流程。
- 内置钱包账户体系与扣费、退款、充值、争议处理能力。
- 实时推送租借、支付、故障、维修等场景通知。
- 提供管理员综合看板，展示用户、设备、订单、财务指标。

架构与技术栈

领域	技术/方案
语言	Java 21
框架	Spring Boot 3.2.x、Spring MVC、Spring Validation、Spring Security (仅做简单 Token 校验)
构建	Maven
数据格式	RESTful JSON
日志 & 监控	Spring Boot Actuator (已引入，可进一步扩展)
存储	当前为内存 Map，方便演示；后续可接入关系型数据库或 NoSQL
推送	简化为站内消息模型，可扩展为短信/微信/消息队列

环境准备

- 操作系统：Windows / macOS / Linux
- JDK：JDK 21
- 构建工具：Maven 3.9+
- IDE：推荐 IntelliJ IDEA / VS Code（需装 Java 相关插件）

快速启动

```
# 1. 拉取代码后进入项目根目录
cd /path/to/carebed-platform

# 2. 编译（跳过测试）
mvn -DskipTests compile

# 3. 运行 Spring Boot
mvn spring-boot:run

# 4. 或打包后运行
mvn -DskipTests package
java -jar target/carebed-platform-0.0.1-SNAPSHOT.jar
```

服务启动后默认监听 `http://localhost:8080`。

配置说明

`src/main/resources/application.yml` 中仅包含基础应用名与端口配置。若要切换端口或集成真实数据源，可新增如下配置：

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/carebed
    username: root
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver
```

同时根据需要开启 Spring Scheduling、消息队列、缓存等组件。

角色与权限矩阵

功能 / 角色	陪护家属 (FAMILY)	运营管理员 (ADMIN)
注册 / 登录	✓	✓
关联患者信息	✓	✕
浏览设备列表	只读	CRUD
设备绑定/解绑	自身租借流程触发	手动干预
发起租借/归还	✓	可代操作
查看订单	仅本人	全量
钱包充值 / 查询	✓	✓（可查看所有用户申诉）
发起费用争议	✓	审核处理
提交报修	✓	✓（可新建及修改状态）
处理报修	✕	✓
查看通知	✓	✓（含管理员渠道）
查看运营看板	✕	✓

功能模块详解

1. 用户与认证

- POST /api/auth/register 创建账号，支持 FAMILY/ADMIN 角色。
- POST /api/auth/login 登录后返回临时会话 Token（有效期 12 小时）。
- POST /api/auth/link-patient 家属可填写住院号/床位号，方便订单关联。
- 数据结构：UserAccount（UUID 主键、BCrypt 加密密码、角色、联系方式、患者关联信息）。

2. 智能设备管理

- 设备注册、更新、删除、绑定、释放、心跳上报、故障上报、远程重启。
- 设备状态枚举：AVAILABLE、IN_USE、MAINTENANCE、OFFLINE。
- 事件流 DeviceEvent 记录每次操作，用于追踪设备生命周期。

3. 租借使用与归还

- `POST /api/rentals/start` 启动租借，绑定设备与用户。
- `POST /api/rentals/{id}/return` 归还设备并完成计费（按小时向上取整）。
- 支持管理员读取全部订单、家属仅查看自身订单。
- 内置超时扫描方法 `scanForOverdue()`，可通过定时任务触发。

4. 消息与通知

- 所有关键动作触发 `Notification`，例如租借成功、超时提醒、扣费通知、维修更新。
- 用户与管理员分别维护收件箱，可手动标记已读。

5. 报修与维修

- `POST /api/repairs` 家属发起报修，上传描述与照片（Base64 或外链）。
- 系统自动关联当前订单与设备，管理员可更新状态：`OPEN` → `IN_PROGRESS` → `RESOLVED`。

6. 我的钱包

- 钱包账户 `WalletAccount` 记录余额，所有交易写入 `WalletTransaction`。
- 支持充值、扣费、退款、争议处理；管理员可调整争议单状态及处理意见。

7. 平台综合管理

- 管理员概览 `GET /api/admin/overview` 展示用户、设备、订单、钱包余额、报修情况等统计。
- `GET /api/admin/analytics` 返回使用趋势与营收趋势，可用于可视化。

核心业务流程

租借流程（家属端）

1. 登录并关联患者信息。
2. 扫码选择设备，调用 `/api/rentals/start`。
3. 系统：
 - 校验设备状态 → 将设备标记为 `IN_USE`。
 - 创建 `RentalRecord`，记录预期结束时间（按期望时长）。
 - 发送租借成功通知给家属和管理员。
4. 使用结束后，家属归还：`/api/rentals/{id}/return`，需确认关锁。
5. 系统：
 - 计算使用时长与费用 → 钱包扣费。

- 释放设备，恢复 AVAILABLE 。
- 发送关锁确认与扣费通知。

超时检测

- 定期执行 RentalService.scanForOverdue()：
 - 找出预期结束时间已过且仍 ACTIVE 的订单。
 - 将订单标记为 OVERDUE ，推送“即将超时/已超时”提醒。

报修流程

1. 家属在订单中发现故障，提交报修。
2. 系统关联设备与订单，通知管理员。
3. 管理员更新状态及处理说明，用户实时接收进度通知。
4. 设备可在维修状态中被下架或安排维护。

钱包争议处理

1. 用户在钱包模块发起费用争议（指定订单号、原因）。
2. 管理员列表查看全部争议，更新状态为 IN_REVIEW / RESOLVED / REJECTED ，填写结论。
3. 对应通知推送给用户，必要时可补发退款交易。

数据模型速览

实体	关键字段
UserAccount	id 、 username 、 passwordHash 、 role 、 linkedPatientId 、 createdAt
Device	id 、 deviceCode 、 ward 、 bedNumber 、 status 、 batteryLevel 、 lastHeartbeat
RentalRecord	id 、 userId 、 deviceId 、 status 、 startedAt 、 expectedEndAt 、 amount
WalletAccount	userId 、 balance 、 updatedAt
WalletTransaction	id 、 userId 、 type (RECHARGE/DEBIT/REFUND/ADJUSTMENT)、 amount 、 orderId
NotificationMessage	id 、 userId 、 type 、 title 、 content 、 read 、 createdAt
RepairTicket	id 、 userId 、 rentalId 、 deviceId 、 description 、 photos 、 status

API 参考

以下为关键接口示例（完整定义参见 `src/main/java/com/carebed/**/controller`）：

认证与用户

POST `/api/auth/register`

Content-Type: `application/json`

```
{
  "username": "family001",
  "password": "Password@123",
  "role": "FAMILY",
  "fullName": "张三",
  "phone": "+8613811112222"
}
```

成功返回：`{"token":"...","role":"FAMILY","displayName":"张三"}`。

设备管理

POST `/api/devices`

Content-Type: `application/json`

```
{
  "deviceCode": "BED-0001",
  "ward": "内科三区",
  "bedNumber": "305-2"
}
```

租借归还

POST `/api/rentals/{id}/return`

X-Auth-Token: <登录返回的 `token`>

Content-Type: `application/json`

```
{
  "conditionNotes": "设备完好",
  "lockConfirmed": true
}
```

若余额不足扣费，接口返回 400 并提示“余额不足，请充值”。

钱包申诉

```
POST /api/wallet/disputes
X-Auth-Token: <token>
Content-Type: application/json

{
  "orderId": "a0f1...",
  "reason": "扣费金额异常"
}
```

管理员处理争议： POST /api/wallet/disputes/{id} ，更新状态与处理说明。

通知策略

触发事件	通知类型	接收方	说明
租借创建	RENTAL_SUCCESS	家属、管理员	提示订单开始
预期超时	RENTAL_EXPIRING	家属	每次扫描发现超时即发送
归还确认	LOCK_CONFIRMED	家属	记录归还完成
钱包扣费	PAYMENT_CHARGED	家属、管理员	通知扣费或退款金额
充值成功	PAYMENT_ALERT	家属	提醒余额变动
报修提交	REPAIR_UPDATE	管理员	鼓励及时处理
报修状态变化	REPAIR_UPDATE	家属	监控维修进度
系统异常/取消	SYSTEM_ALERT	相关用户	如订单被取消

通知默认存储在内存消息仓库，可扩展至消息队列、短信、推送平台。

测试与质量保障

- 单元测试： mvn test
- 编译检查： mvn -DskipTests compile
- 建议引入以下实践：
 - 使用 Testcontainers / H2 模拟真实数据库。

- 在租借、扣费等关键流程加入集成测试。
- 接入 SonarQube 或 SpotBugs 做静态扫描。

运维与部署建议

1. **外部化配置**：使用 `application-prod.yml` 和环境变量，避免硬编码密码。
2. **日志管理**：集成 ELK/EFK 并对关键操作（扣费、故障）写入审计日志。
3. **持久化存储**：迁移至数据库后，为核心表加上乐观锁或悲观锁确保一致性。
4. **高可用**：
 - 使用 Nginx/负载均衡进行流量分发。
 - 结合 Spring Cloud 或 Kubernetes，实现弹性伸缩。
5. **监控告警**：借助 Actuator + Prometheus + Grafana 实现性能与业务指标监控。

常见问题

1. **为什么没有看到持久化数据？**
 - 当前 Demo 使用内存 Map 保存数据，重启后会重置。
2. **如何扩展 Token 机制？**
 - 可替换为 JWT（Spring Security + jjwt），或改用 OAuth2/OpenID Connect。
3. **如何接入真实支付？**
 - 目前钱包模块为虚拟账户，为接入第三方支付需新增回调接口与账务对账表。
4. **蓝牙开锁如何对接？**
 - 在租借开始前调用第三方蓝牙服务 API，并将其结果记录到 `DeviceEvent`。

未来规划

- **设备物联网化**：对接硬件心跳、GPS、电量、锁控等实时数据。
- **消息多渠道**：集成短信、微信公众号、小程序模板消息等。
- **多医院支持**：引入组织维度的多租户设计，实现跨院区管理。
- **计费策略升级**：支持按次、按天、阶梯价、优惠券、押金等模式。
- **风险控制**：增加设备盗损检测、异常用电报警、人机巡检流程。
- **数据分析**：引入大屏可视化，实时展示床位分布、使用热力图、维修 SLA。

如需更多帮助或希望拓展特定模块，请联系项目维护者或提交 Issue。祝使用顺利！

- 1. 安装 JDK 21 与 Maven 3.9 及以上版本。
- 2. 本示例使用内存存储，无需额外配置密钥或数据库。
- 3. 编译项目：`mvn -DskipTests compile`
- 4. 启动服务：`mvn spring-boot:run`

默认访问地址：`http://localhost:8080`

功能模块

- **用户与认证**：支持陪护家属与管理员注册、登录，使用 `X-Auth-Token` 请求头访问受保护接口，家属可关联患者住院号或床位号。
- **智能设备管理**：设备注册、信息维护、绑定患者、心跳与电量上报、故障上报及远程重启，保留事件轨迹。
- **租借流程**：扫码取床（模拟蓝牙开锁）、使用计费、归还自动扣费，支持超时检测与运营监控。
- **消息通知**：实时推送租借成功、超时提醒、关锁确认、扣费提示、报修进度等通知，提供用户与管理员收件箱。
- **报修与维修**：家属可提交描述与照片，系统自动关联设备与订单；管理员端跟踪处理状态并更新用户。
- **我的钱包**：余额查询、充值、扣费、退款、费用争议申诉，运营端可人工审核处理。
- **平台运营管理**：管理员查看用户、设备、订单、财务数据，获取设备在线率/使用率/故障情况及收入趋势。

核心 API

模块	方法	路径	说明
认证	POST	<code>/api/auth/register</code>	多角色注册
认证	POST	<code>/api/auth/login</code>	登录并获得 <code>X-Auth-Token</code>
家属	POST	<code>/api/auth/link-patient</code>	关联患者住院号/床位号
设备	POST	<code>/api/devices</code>	新增设备
设备	POST	<code>/api/devices/{id}/faults</code>	提交故障上报
租借	POST	<code>/api/rentals/start</code>	创建租借订单
租借	POST	<code>/api/rentals/{id}/return</code>	归还并结算
消息	GET	<code>/api/messages</code>	当前用户消息列表
报修	POST	<code>/api/repairs</code>	提交报修工单

模块	方法	路径	说明
钱包	GET	/api/wallet	钱包余额与流水
管理	GET	/api/admin/overview	平台指标概览

更多接口可参考对应控制器文件。

认证与默认账号

- 所有需要鉴权的接口使用 `X-Auth-Token` 请求头。
- 系统启动时会自动初始化一个管理员账号：用户名 `admin` ，密码 `Admin@123` 。

测试与维护

- 单元测试：`mvn test`
- 查看依赖更新：`mvn versions:display-dependency-updates`
- 当前数据存储基于内存集合，后续可替换为数据库或消息队列实现。

未来规划

- 与真实蓝牙开锁服务对接，在 `/api/rentals/start` 中下发指令。
- 引入持久化存储、消息队列与分布式事务管理。
- 使用 Spring Scheduling 定时执行 `RentalService.scanForOverdue()` 处理超时订单。